Copyright (c) 2026 Moeez Raza

MR WS Handler Fast API

Websocket Handler for Fast API developed by Moeez Raza

Copyright (c) 2026 Moeez Raza

MR WS Handler for FastAPI

1. Introduction

A lightweight, reusable WebSocket handler for FastAPI – developed by Moeez Raza.

The goal of this project is to make using Web-sockets in Fast-API **easier, simpler, and faster**. Stop copying connection management code between projects. Just install, import, and focus on your real-time logic.

2. Features

- ✓ **Simple connection manager** – automatically handles **accept**, store, and disconnect.
- ✓ **Broadcast to all connected clients** – one line to send a message to everyone.
- ✓ **Send to specific client ID** – direct messaging with integer or string IDs.
- ✓ **Room support** – group clients into rooms and send messages to a room only.
- ✓ **Multiple message types** – **str**, **bytes**, **dict** (auto-converted to JSON via **send_message**).
- ✓ **Flexible endpoint modes** – choose **mode="str"**, **"bytes"**, or **"dict"** for incoming messages.
- ✓ **Life-cycle decorators** – **@onopen** and **@onclose** callbacks for connection events.
- ✓ **Ready-to-use HTML/JS test client** – full UI to test connections, client IDs, and rooms.
- ✓ **Fully async** – built for high-performance FastAPI apps.
- ✓ **Global connection manager** – a single **connectionManager** instance, accessible anywhere.
- ✓ **Reusable** – publish to PyPI and **pip install** in any project.

3. Installation

```
pip install mr_wshandler_fastapi
```

Or install directly from GitHub:

```
pip install git+https://github.com/moeezraza35/mr_wshandler_fastapi.git
```

4. Quick Start

4.1. Run the demo (easiest way to test)

Clone the repository and run the included demo server:

```
git clone https://github.com/your-username/mr_fastapi_wshandler.git
cd mr_fastapi_wshandler
python -m venv venv
source venv/bin/activate  # or venv\Scripts\activate on Windows
pip install -r requirements.txt
python demo/main.py
```

Open your browser at http://localhost:8000. You'll see a full HTML/JS test client that lets you:

- Connect to the Web-socket
- Send text or JSON messages
- Set a custom client ID
- Join / leave rooms
- See received messages in real time

Copyright (c) 2026 Moeez Raza

5. API Reference

5.1. WSHandler

The main entry point for your Web-socket endpoint.

- **WSHandler()** – creates a new handler instance. It automatically uses the global **connectionManager** (no need to create one manually).
- **@WSHandler.endpoint(mode="str")** – decorator for your WebSocket route.
  - **mode** can be **"str"** (default), **"bytes"**, or **"dict"**.
  - It determines how incoming messages are received and passed to your callback.
  - Your callback function will receive the decoded data (string, bytes, or dict).
- **@WSHandler.onopen** (optional) – decorator to register a callback that runs whenever a new Web-socket connection is accepted.

```
@ws.onopen
async def on_open(websocket: WebSocket):
    print(f"New client connected: {websocket}")
```

- **@WSHandler.onclose** (optional) – decorator to register a callback that runs when a WebSocket connection is closed (client- or server- initiated).

```
@ws.onclose
async def on_close(websocket: WebSocket):
    print(f"Client disconnected")
```

Note: The **onopen** and **onclose** callbacks are called after the connection has been added to / removed from the manager. You can access **connectionManager** inside them to inspect active connections.

5.2. connectionManager (global instance)

The singleton that manages all active Web-Socket connections. It is created automatically when you instantiate **WSHandler()**.

5.2.1. Core Connection Management

Method	Description
async connect(websocket: WebSocket)	Accepts the WebSocket and stores the connection. Called automatically by **@ws.endpoint**.
async disconnect(websocket: WebSocket)	Closes the WebSocket (server- initiated) and removes it from the manager.
async remove_connection(websocket: WebSocket)	Removes a connection without closing it – used when the client has already disconnected.
async set_client_id(websocket: WebSocket, clientId: int | str)	Assigns a unique identifier to a connection (e.g., user ID, session ID) default is **0**.

Copyright (c) 2026 Moeez Raza

5.2.2. Sending Messages

Method	Description
async send_message(message: str | bytes | dict, websocket: WebSocket)	Core sender – automatically handles the message type (string → **send_text**, bytes → **send_bytes**, dict → **send_json**). Used internally by all other send methods.
async send_message_to_connection(...)	Alias for **send_message**. Sends a message to a specific connection.
async send_message_to_client_id(message, clientId: int | str)	Looks up the connection by **clientId**, then calls **send_message** to deliver the message.
async broadcast(message: str | bytes | dict)	Iterates over all active connections and calls **send_message** for each one.

5.2.3. Room Management

Rooms allow you to group connections and send messages only to members of a specific room.

Method	Description
add_client_id_to_room(clientId: int | str, room: str)	Adds a client (by ID) to a room.
remove_client_id_from_room(clientId: int | str, room: str)	Removes a client from a room.
add_connection_to_room(websocket: WebSocket, room: str)	Adds a connection (by Web-socket object) to a room.
remove_connection_from_room(websocket: WebSocket, room: str)	Removes a connection from a room.
async send_message_to_room(message: str | bytes | dict, room: str)	Iterates over all connections in the given room and calls **send_message** for each one.

5.2.4. Example:

```
# Inside your WebSocket callback
connectionManager.add_connection_to_room(websocket, "general")
await connectionManager.send_message_to_room("Hello room!", "general")
```

5.2.5. Typical Usage Flow

```
from mr_wshandler import WSHandler, connectionManager

ws = WSHandler()

@ws.onopen
async def onopen(websocket):
    print("New connection")

@ws.endpoint(mode="dict")
async def onMessage(data: dict, websocket: WebSocket):
    if "room" in data:
        # Join a room
        connectionManager.add_connection_to_room(websocket, data["room"])
        await connectionManager.send_message_to_room(f"{websocket} joined", data["room"])
    else:
        # Broadcast to all
        await connectionManager.broadcast(data)
```

Copyright (c) 2026 Moeez Raza

6. Testing

Run the included demo server to test all features:

```
python demo/main.py
```

Then open http://localhost:8000 in your browser. The HTML test client lets you connect, send messages, set client IDs, and join/leave rooms – all interacting with your Web-Socket module in real time.

7. Contributing

Contributions, bug reports, and feature requests are warmly welcome! Feel free to open issues or submit pull requests.

If you fork this repository, please include a note in your **README** or **documentation** that your project is based on **mr_wshandler_fastapi** by Moeez Raza. A simple line like _*"Originally forked from mr_fastapi_wshandler by Moeez Raza"*_ is appreciated.

8. License

This project is licensed under the MIT License – see the LICENSE.md file for details.

The MIT License already requires that the original *copyright notice (© Moeez Raza)* be retained in all copies or substantial portions of the software.

9. References

- FastAPI WebSocket Documentation
- Python WebSockets